

RAPPORT TECHNIQUE DE PROJET

SYSTEME DE GESTION DE RESTAURANT ET DE STOCKS : GASTROPOS

Rédigé par : Raye Kitou Fatima

Spécialité : Génie Logiciel

Date : Juin 2026

PREAMBULE & INTRODUCTION

Dans le secteur de la restauration, la synchronisation en temps réel entre la prise de commande à la caisse (Front-End), la visibilité de la file d'attente en cuisine (Dashboard) et la consommation des matières premières (Gestion des stocks) représente un défi logistique majeur. Les erreurs de saisie, les retards de communication et le manque de visibilité sur les stocks d'ingrédients mènent inévitablement à des pertes financières et à une baisse de la satisfaction client.

Le projet **GastroPOS** répond précisément à cette problématique en proposant une architecture d'application web légère, robuste et réactive. Développée avec le micro-framework **Flask (Python)** pour le backend et **MySQL** pour la persistance des données, l'application centralise le flux d'activité journalier. Ce rapport technique détaille l'analyse des besoins, la conception de la base de données, l'implémentation de la logique métier et les choix d'interface utilisateur basés sur le framework **Tailwind CSS**.

1. ANALYSE DES BESOINS ET FONCTIONNALITES

L'application GastroPOS s'articule autour de trois modules interconnectés :

- Le Menu Interactif (Caisse/Prise de commande)** : Une interface épurée permettant au personnel ou au client de sélectionner dynamiquement des menus (ex: Chou Épicé, Allocò, Burger), d'ajuster les quantités dans un panier virtuel, de renseigner l'identifiant de la table (ou référence client) et de valider la commande via des requêtes asynchrones HTTP **POST** (Fetch API).
- Le Flux d'Activité en Temps Réel (Dashboard)** : Une interface d'administration permettant de suivre l'évolution des commandes. Chaque commande y est listée chronologiquement avec son identifiant unique, la table associée, la date/heure précise, **la liste explicite des menus pris**, le montant total calculé et un sélecteur dynamique permettant de modifier instantanément le statut de la commande (**En attente**, **Servi**, **Payé**).
- Le Suivi Automatisé des Stocks d'Ingrédients** : Un module d'arrière-plan analytique. Chaque menu du restaurant est lié à une "recette" spécifiant les

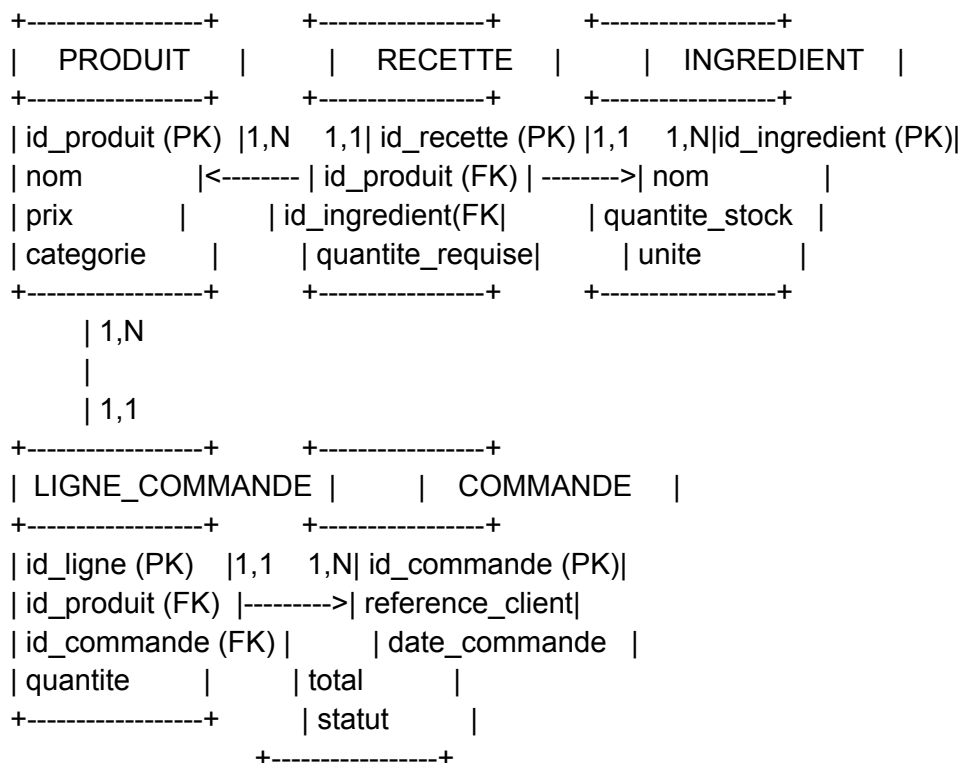
composants nécessaires (ex: un Burger consomme 1 pain et 1 steak). Lors de la validation d'une commande, le système déduit automatiquement les quantités requises du stock global et applique une alerte visuelle (couleur rouge) dans l'interface si un ingrédient descend sous le seuil critique de 10 unités.

2. ARCHITECTURE TECHNIQUE & BASE DE DONNEES

Pour assurer la cohérence et l'intégrité des calculs (notamment pour éviter les flux à zéro ou les désynchronisations de prix), le modèle relationnel a été normalisé en **3NF (Troisième Forme Normale)**, séparant les entités de commandes globales des lignes de détails.

Modèle Conceptuel de Données (MCD)

- **produit (1,N) <---- (1,1) recette (1,1) ----> (1,N) ingredient** : Un produit possède plusieurs ingrédients via la table des recettes, spécifiant la quantité requise.
- **commande (1,N) <---- (1,1) ligne_commande (1,1) ----> (1,N) produit** : Une commande regroupe plusieurs produits avec leurs quantités respectives.



3. IMPLEMENTATION DU BACKEND (FLUX ET SECURITE LOGIQUE)

La logique backend développée dans `app.py` repose sur une gestion rigoureuse des transactions SQL. Le point le plus sensible est la route `/passer_commande`. Pour éviter qu'une rupture de stock ou un plantage SQL ne crée une commande fantôme, le mécanisme intègre un bloc `try/except` avec `conn.rollback()` en cas d'échec :

1. **Initialisation** : La commande est créée à l'état `EN ATTENTE` avec un total temporaire à `0.00`.
2. **Calcul et Liaison** : Pour chaque article du panier, le système interroge la table `produit` pour récupérer le prix unitaire réel (en nettoyant les caractères non numériques via des expressions régulières `re.sub(r'\D', ...)`). Le prix cumulé est injecté dans `ligne_commande`.
3. **Déduction des Stocks** : Une sous-requête récupère la recette du produit et soustrait (`UPDATE ingredient SET quantite_stock = quantite_stock - ...`) la matière première proportionnellement à la quantité commandée.
4. **Validation** : Le montant final calculé est mis à jour dans la table `commande` et la transaction est définitivement validée via `conn.commit()`.

Pour l'affichage du Dashboard, afin d'optimiser les performances et d'éviter la multiplication des requêtes dans les boucles (problème du N+1 Query), une jointure SQL avancée utilisant `GROUP_CONCAT` rassemble l'ensemble des menus pris en une seule ligne textuelle structurée par commande, garantissant un affichage instantané.

4. CONCEPTION DE L'INTERFACE UTILISATEUR (UI/UX)

L'identité visuelle de GastroPOS utilise une thématique haut de gamme baptisée "**Glow-Mix**", s'appuyant sur les classes utilitaires de **Tailwind CSS v4**.

- **Charte graphique** : Palette de couleurs basée sur des tons chauds et ambrés (`#78350f` à `#ea580c`), contrastée par un fond neutre en pierre (`bg-stone-100`). Les indicateurs financiers utilisent un vert émeraude percutant pour le chiffre d'affaires.
- **Dynamisme visuel** : Les cartes statistiques intègrent des bordures en dégradé animé (`@keyframes gradientShift`) s'activant au survol de la souris (`hover`), offrant une expérience interactive moderne et professionnelle.
- **Ergonomie du flux** : L'utilisation de badges colorés de statuts (Jaune pour l'attente, Bleu pour le service, Vert pour le paiement) permet au gestionnaire d'analyser l'état de la salle en un seul coup d'œil. Le changement de statut via l'événement `onchange="this.form.submit()"` élimine les clics superflus.

CONCLUSION

GastroPOS démontre l'efficacité de l'association d'un backend Python/Flask structuré et d'une base de données relationnelle correctement normalisée. Les verrous logiques implémentés sécurisent l'alignement strict entre les menus choisis par les clients, les calculs de facturation affichés sur le tableau de bord et l'état réel des stocks d'ingrédients en cuisine. Le projet remplit l'ensemble des exigences fonctionnelles et architecturales dictées par le cahier des charges.